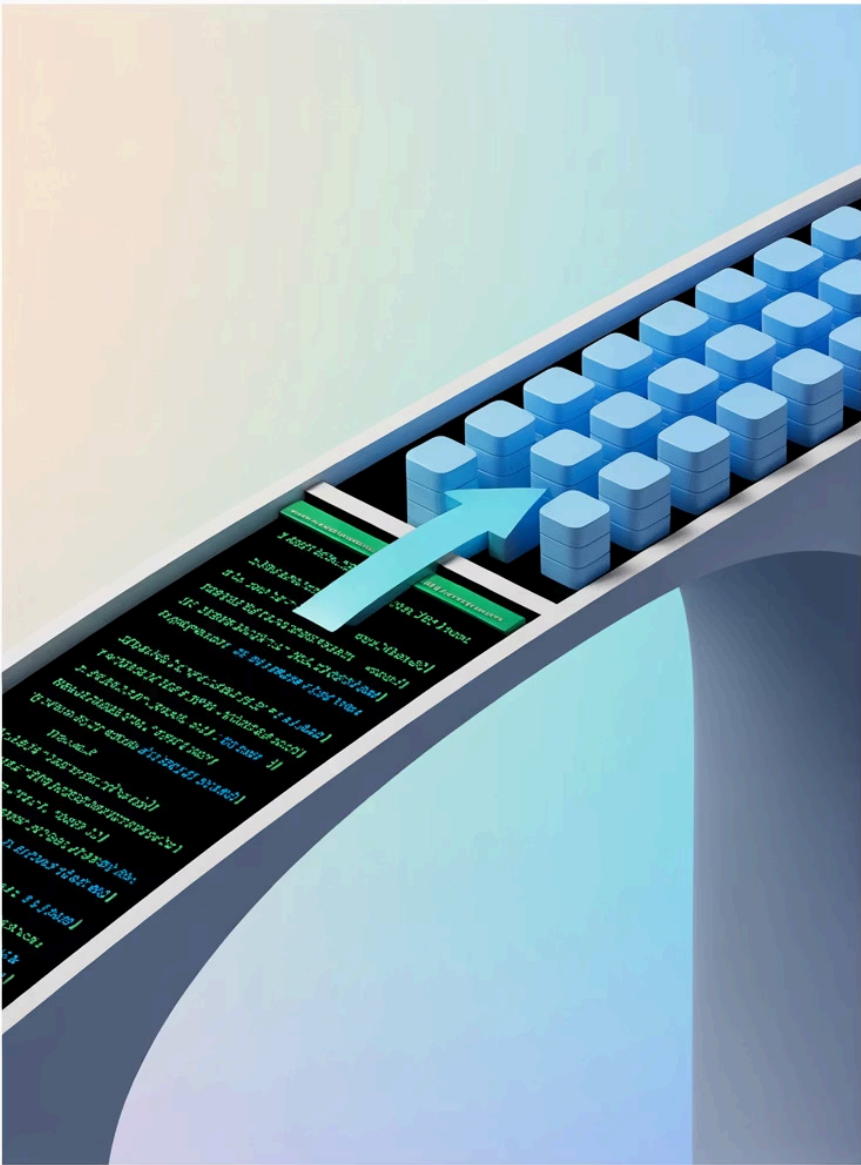




Jakarta Persistence API (JPA)

Un puente entre objetos Java y bases de datos relacionales

¿Qué es JPA?

JPA (anteriormente Java Persistence API, ahora Jakarta Persistence API) es una especificación estándar de Java para el mapeo objeto-relacional (ORM) y la persistencia de datos.

En términos sencillos, JPA es un puente que permite mapear objetos Java a tablas de bases de datos relacionales, gestionar el ciclo de vida de objetos persistentes y abstraer la base de datos subyacente.



Beneficios principales de JPA

Mapeo objeto-relacional

Permite trabajar con objetos Java (POJOs) en lugar de escribir SQL directamente para cada operación (INSERT, SELECT, UPDATE, DELETE).

Gestión del ciclo de vida

Maneja la creación, lectura, actualización y eliminación de objetos de la base de datos de manera transparente.

Abstracción de base de datos

Facilita el cambio entre bases de datos (MySQL, PostgreSQL, H2) con cambios mínimos en el código Java.

Puntos clave sobre JPA

Especificación, no implementación

JPA es solo un conjunto de interfaces y reglas. Necesitas un proveedor de JPA (como Hibernate, EclipseLink, OpenJPA) para utilizarlo en tu aplicación.

ORM (Object-Relational Mapping)

JPA encapsula el patrón ORM, resolviendo el "desajuste" entre el paradigma orientado a objetos y el paradigma relacional de las bases de datos.

Simplifica el acceso a datos

Reduce drásticamente el código repetitivo y propenso a errores que tendrías que escribir usando JDBC directamente.

Configuración de JPA

Basado en anotaciones o XML

Puedes configurar tus entidades y sus mapeos a tablas usando:

- Anotaciones en tus clases Java (forma más común y recomendada)
- Archivos XML (como persistence.xml)

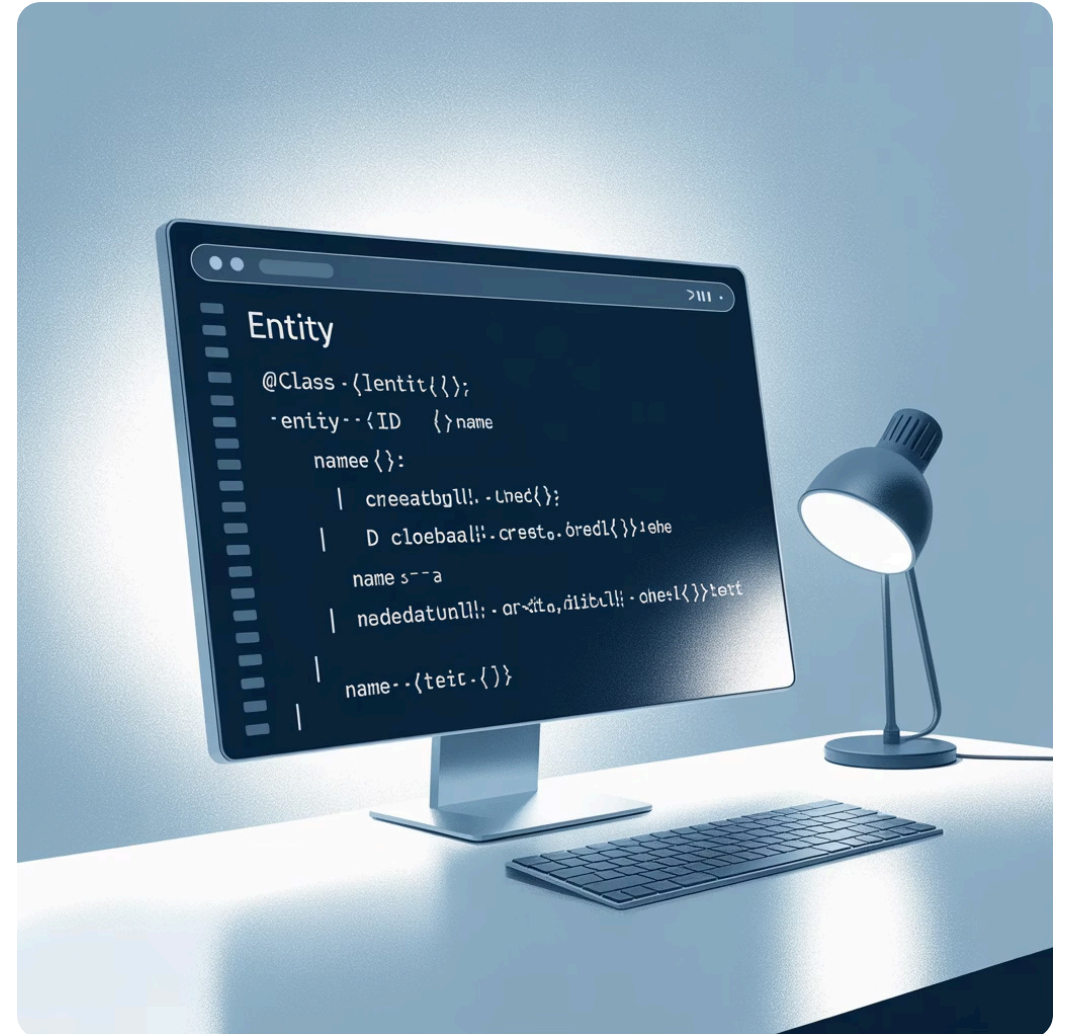


Componentes principales de JPA



Entidades (Entities)

- Clases Java POJO que representan tablas en la base de datos
- Marcadas con la anotación @Entity
- Contienen campos que se mapean a columnas de la tabla
- Deben tener una clave primaria, mapeada con @Id
- Pueden tener relaciones con otras entidades (@OneToOne, @OneToMany, @ManyToOne, @ManyToMany)



EntityManager

1

`persist()`

Guarda una nueva entidad en la base de datos

2

`find()`

Busca y devuelve una entidad por su clave primaria

3

`merge()`

Actualiza una entidad existente o la hace persistente si es nueva

4

`remove()`

Elimina una entidad de la base de datos

5

`refresh()`

Recarga el estado de una entidad desde la base de datos

La interfaz central para la interacción con la base de datos que gestiona el ciclo de vida de las entidades.

EntityManagerFactory

Es una fábrica para crear instancias de EntityManager:

- Objeto costoso de crear, típicamente se crea una sola vez por aplicación
- Se reutiliza durante toda la vida de la aplicación
- Se obtiene usando:
`Persistence.createEntityManagerFactory("nombreUnidadPersistencia")`



Unidad de Persistencia y persistence.xml

Unidad de Persistencia

Define un conjunto de entidades que se gestionan juntas, especificando la configuración de la conexión a la base de datos, las clases de entidad a mapear y las propiedades del proveedor de JPA.

persistence.xml

Archivo de configuración estándar de JPA ubicado en META-INF/persistence.xml que contiene la definición de una o más unidades de persistencia.



JPQL (Jakarta Persistence Query Language)

Es un lenguaje de consulta orientado a objetos definido por JPA:

- Similar a SQL, pero opera sobre entidades y sus atributos en lugar de tablas y columnas
- Permite escribir consultas portables que funcionan con diferentes bases de datos
- No requiere cambios en el código de la consulta al cambiar de base de datos

 Ejemplo JPQL:

```
SELECT p FROM Persona p WHERE p.edad > 30
```

En lugar de SQL:

```
SELECT * FROM Persona WHERE edad > 30
```

Proveedores de JPA



Hibernate

El más utilizado y maduro. Ofrece funcionalidades adicionales más allá de la especificación JPA.



EclipseLink

La implementación de referencia de JPA. Desarrollada por la Fundación Eclipse.



OpenJPA

Un proveedor de Apache. Buena integración con otros proyectos Apache.



TopLink

De Oracle, basado en EclipseLink. Integración nativa con productos Oracle.

Ventajas de usar JPA

Productividad

Reduce significativamente la cantidad de código necesario para interactuar con bases de datos.

Mantenibilidad

Código más limpio y orientado a objetos, facilitando el mantenimiento a largo plazo.

Portabilidad

Permite cambiar entre diferentes bases de datos con cambios mínimos en el código.

Rendimiento

Los proveedores de JPA implementan optimizaciones como caché y carga perezosa para mejorar el rendimiento.



Resumen: JPA en pocas palabras

JPA te proporciona una API estándar y un modelo de programación para la persistencia de objetos en bases de datos relacionales, permitiéndote enfocarte en la lógica de negocio en lugar de los detalles de SQL y la gestión de la base de datos.